



Cloud Computing & Big Data Analytics

DAY 01 — Introduction to Cloud Computing

Digital Science Graduate Programme | Lesson Notes

Today's Agenda

- LESSON 1 — Introduction to Cloud Computing (~30 min)
 - LESSON 2 — Cloud Computing Architecture (~30 min)
 - LESSON 3 — Virtualization and Containerization (~25 min)
 - WRAP-UP — Q&A + Next Steps (~5 min)
-
- Prerequisites: Basic Python familiarity | General IT literacy
 - Session Level: Introductory Graduate

Why Cloud Computing Matters to Data Scientists

- Your models will train on cloud GPUs
- Your pipelines will run on cloud-managed Kubernetes clusters
- Your datasets (100s of GBs) will live in cloud object storage
- Your global colleagues will hit your API endpoints in real time
- Cloud-native data science roles command €120k+ salaries in Europe
- "Boring doesn't pay €120k/year. Cloud-native data science does."

 *Cloud is not just a tool you use — it is the environment you will live in for the next decade.*

LESSON 1

Introduction to Cloud Computing

What Is Cloud Computing? — The NIST Definition

Official: "...on-demand network access to a shared pool of configurable computing resources that can be rapidly provisioned and released with minimal management effort." — NIST SP 800-145

Plain English: Renting someone else's powerful computers over the internet, paying only for what you use, and scaling up or down whenever you want.

Analogy: Think of electricity. No bakery builds its own power plant. They plug in and pay per kWh. Cloud is the same idea — but for compute, storage, and software.

A Brief History of Cloud Computing

Year	Event
1960s	Time-sharing — John McCarthy proposes computing as a public utility
1999	Salesforce launches — first major SaaS (Software-as-a-Service) product
2006	AWS launches S3 + EC2 — the modern cloud era officially begins
2008	Google App Engine — PaaS model at scale
2010	Microsoft Azure — major enterprise vendor goes cloud-first
Today	\$600B+ global market, ~20%/year growth. Big Three: AWS, Azure, GCP

 *Data science workloads are one of the fastest-growing categories in cloud spending.*

The 5 Essential Characteristics (NIST) — Part 1 of 2

- ① **On-Demand Self-Service:** Spin up a 32-core server in Singapore with a few clicks. No tickets, no IT team, no waiting.
- ② **Broad Network Access:** Resources accessible over the internet from any device — laptop, phone, tablet, Raspberry Pi — using standard protocols.
- ③ **Resource Pooling:** Multi-tenancy: provider serves many customers simultaneously. Resources dynamically assigned and reassigned. You don't know which physical server you're on, and you don't need to.

The 5 Essential Characteristics (NIST) — Part 2 of 2

④ **Rapid Elasticity:** Scale out (add more) or scale in (reduce) extremely quickly — sometimes automatically. Netflix spins up thousands of servers for peak traffic, then scales back down.

⑤ **Measured Service:** Automatic billing for exactly what you use. Pay-as-you-go. No more €50,000 servers running at 3% capacity for five years.

 *If a service is missing even ONE of these five characteristics, it is not truly cloud — it is just someone's server with a nice dashboard.*

The Three Cloud Service Models

IaaS / PaaS

- IaaS — Infrastructure as a Service
 - Raw compute, storage, networking
 - You manage: OS, middleware, runtime, app
 - Maximum control, maximum responsibility
 - Examples: AWS EC2, Azure VMs, GCE
-
- Data Science: Rent GPU → install CUDA + Python + PyTorch yourself
-
- SaaS — Software as a Service
 - Fully managed software via browser/API
 - Provider handles everything
 - Examples: Snowflake, Weights & Biases, Google Workspace
 - Data Science: Log ML experiments — no server management

PaaS / SaaS

- PaaS — Platform as a Service
 - Infrastructure managed for you
 - You just bring your code and data
 - Examples: Databricks, Google App Engine, AWS Elastic Beanstalk
-
- Data Science: Write Spark job → Databricks handles cluster, scaling, logging
-
- The Hierarchy:
 - More Convenience ▲
 - SaaS ← You use the app
 - PaaS ← You bring your code
 - IaaS ← You manage most things
 - More Control ▼

The Four Cloud Deployment Models

Public & Hybrid

-  Public Cloud
 - Owned by third-party, shared by many customers
 - Most cost-effective option
 - AWS, Azure, GCP
 - Your data lives on their hardware
-
-  Hybrid Cloud
 - Combination of public + private cloud
 - Sensitive/critical workloads stay private
 - Overflow/burst workloads go public
 - Most common model in large enterprises

Private & Multi-Cloud

-  Private Cloud
 - Operated solely for one organisation
 - On-premises or third-party hosted
 - Higher cost, higher control
 - Required for: banking, healthcare, defence
-
-  Multi-Cloud
 - Services from multiple providers simultaneously
 - Example: Data on AWS S3 + Train on GCP TPUs + Deploy on Azure
 - Key benefit: avoids vendor lock-in
 - Growing adoption in large tech companies

Cloud Provider Cost: 1 TB Storage + 3× Full Monthly Retrieval

Provider	Storage/mo	Egress/mo	Total/mo
AWS S3 Standard	\$23.00	\$27.00	~\$50.00
Google Cloud Storage (Standard)	\$20.00	\$24.00	~\$44.00
Azure Blob Storage (Hot Tier)	\$18.40	\$27.00	~\$45.40

💡 *Formula: $Total = (storage_per_gb \times total_gb) + max(0, egress_gb - free_tier) \times egress_rate$ | Egress fees often exceed storage costs!*

Key Takeaways

Lesson 1 — Introduction to Cloud Computing

1

Cloud = on-demand, scalable, measured compute. NIST's 5 characteristics are the gold standard for what 'cloud' actually means.

2

IaaS / PaaS / SaaS is a control-vs-convenience tradeoff. Choose the layer that matches your team's skills and needs.

3

The Big Three are AWS, Azure, and GCP. Egress fees are the most common billing surprise.

4

Deployment model choice (public/private/hybrid/multi) is driven by cost, compliance, and latency requirements.

LESSON 2

Cloud Computing Architecture

Cloud Architecture — The Big Picture

- Architecture answers: how does the cloud actually work under the hood?
- Understanding architecture means you can design systems, not just use them

Foundation: The Client-Server Model — the bedrock of all computing since the 1960s

Client: Any device or application that requests a service or resource

Server: Any machine or process that provides that service or resource

- In cloud, the 'server' is a distributed, virtualised, elastically-scaled collection of resources — but the fundamental request-response dynamic is unchanged
- Example: Your laptop calls the Spotify API (client) → Spotify's GCP infrastructure responds (server)

Core Component: Compute

Virtual Machines (VMs): Software-emulated computers on physical hardware. Full OS. Minutes to start. Examples: AWS EC2, Azure Virtual Machines

Containers: Lightweight, portable app packages sharing the host OS kernel. Milliseconds to start. 10–100× smaller than VMs. Examples: AWS ECS, Google Kubernetes Engine (GKE)

Serverless Functions: Code runs on-demand with zero server management. Pay per invocation. Examples: AWS Lambda, Google Cloud Functions, Azure Functions

Data Science relevance: Serverless functions are increasingly used for ML model inference endpoints — auto-scale to zero when not in use

Core Component: Storage

Object Storage: Files as discrete objects with metadata. Infinitely scalable. Cheapest. Use for: training datasets, model checkpoints, logs. AWS S3, GCP Cloud Storage, Azure Blob Storage

Block Storage: Fixed-size blocks — like a traditional hard drive. Used as the disk for VMs. Faster, more expensive. AWS EBS, Azure Managed Disks

File Storage: Shared file systems accessible by multiple compute instances simultaneously — like a networked drive. AWS EFS, Azure Files

Rule of thumb: Start with object storage for your ML data. Upgrade to block storage only if you need very low latency I/O.

Core Component: Networking

VPC (Virtual Private Cloud): Your own logically isolated network inside the cloud. Everything sensitive goes inside a VPC.

Subnets: Public subnets communicate with the internet. Private subnets are for databases, ML services, internal APIs — never publicly accessible.

Security Groups: Virtual firewalls controlling inbound and outbound traffic to cloud resources.

CDN (Content Delivery Network): Geographically distributed edge servers cache content close to users. User in Tokyo gets data from Tokyo, not Virginia. AWS CloudFront, Cloudflare, Fastly.

💡 *CDN latency matters: a Tokyo user hitting a Virginia server adds 200ms+ of delay. For ML APIs, that is unacceptable.*

Core Components: Load Balancing & IAM

Load Balancer: Distributes incoming requests across multiple server instances. Prevents single server overload. Automatically routes away from failed instances in <30 seconds.

ML relevance: Your inference API receives 10,000 req/sec during peak — load balancer spreads these across 20 model servers evenly.

IAM (Identity & Access Management): Controls who (authentication) can do what (authorisation). AWS IAM, GCP IAM, Azure Entra ID.

Security note: Most cloud security incidents do NOT come from hacking — they come from IAM misconfiguration. Learn IAM early.

API Gateway: Single entry point for all client requests. Handles authentication, rate limiting, logging, routing. AWS API Gateway, GCP Cloud Endpoints.

Regions and Availability Zones

Region: A distinct geographic area. Examples: eu-west-1 (Ireland), us-east-1 (N. Virginia), ap-southeast-1 (Singapore)

Availability Zone (AZ): One or more discrete data centres within a region, with independent power, cooling, and networking. Each region typically has 2–6 AZs.

AWS scale (2024): 33 geographic regions, 105 availability zones globally.

Data sovereignty: GDPR requires EU citizen data stays within EU regions.

Latency: Deploying in eu-west-1 adds 200ms+ delay for Asian users.

Disaster recovery: Distribute across multiple AZs — single data centre failure won't take down your service.

 *Always deploy production ML systems across at least 2 Availability Zones.*

Key Takeaways

Lesson 2 — Cloud Architecture

1

Every ML production system uses: VPC, subnets, IAM, load balancer, storage, and an API gateway. Learn these components first.

2

Load balancers route away from failed servers automatically in under 30 seconds — this is what makes cloud systems 'fault-tolerant'.

3

IAM misconfiguration is the #1 cause of cloud security incidents. Master IAM before anything else.

4

Regions and AZs matter for latency, compliance (GDPR), and resilience. Always deploy across multiple AZs.

LESSON 3

Virtualization and Containerization

The Problem Both Technologies Solve

- "It works on my machine" — The classic developer nightmare
- Scenario: You build a perfect ML pipeline on your dev machine.
- Colleague runs it → crashes immediately
- Production server → crashes again
- Manager's laptop for the demo → kernel panic

Root cause: Different OS versions, different library versions, different system configs — software was not portable.

Cost: Millions in lost engineering time per year across the industry.

Two solutions emerged: Virtualisation (abstract the hardware) and Containerisation (abstract the OS)

What Is a Virtual Machine (VM)?

Definition: A software-emulated computer that runs on top of physical hardware, with its own OS, CPU allocation, RAM, and disk — but it is not real hardware. It is a software abstraction.

Example: One 64-core/512 GB server can comfortably run 20 independent 4-core/16 GB VMs simultaneously.

The Hypervisor: Software that makes VMs possible. Sits between physical hardware and VMs. Manages resource allocation and isolation. Also called VMM (Virtual Machine Monitor).

Two types: Type 1 (bare-metal) runs directly on hardware — used by cloud providers. Type 2 (hosted) runs as an app on an OS — used for development and testing.

Type 1 vs Type 2 Hypervisors

Type 1 (Production)

- TYPE 1 — Bare-Metal Hypervisor
- Runs directly on physical hardware
- No host OS in between
- Faster, more efficient, more secure
- Used by cloud providers at massive scale
- Examples:
- VMware ESXi
- Microsoft Hyper-V (used by Azure)
- KVM — Kernel-based VM (AWS + GCP)
- Xen (early Amazon)
-  Faster  More Secure  AWS/Azure/GCP

Type 2 (Development)

- TYPE 2 — Hosted Hypervisor
- Runs as an application on a host OS
- Used for desktop dev, testing, education
- Performance overhead from host OS layer
- Examples:
- Oracle VirtualBox (free, open-source)
- VMware Workstation
- Parallels Desktop (macOS)
-  Easy to install  Performance overhead

Virtual Machines vs Physical Servers

Feature	Physical Server	Virtual Machine
Setup time	Days / Weeks (hardware procurement)	Minutes (via API or console)
Cost model	CapEx — large upfront purchase	OpEx — pay-as-you-go
Isolation	Complete (dedicated hardware)	Strong (hypervisor isolation)
Density	1 workload per server	Multiple workloads per server
Snapshot/Backup	Complex, time-consuming	Built-in — snapshot in seconds
Portability	None (hardware-tied)	High (move VMs between hosts)
Resource waste	High (10–15% utilisation typical)	Low (multiple VMs share resources)

💡 *VMs enable cloud providers to sell compute at low cost by maximising server utilisation.*

What Is Containerisation?

Definition: A container packages your application and its dependencies into a single, portable, lightweight unit — but shares the host OS kernel. No guest OS inside the container.

VM analogy: A VM is a complete apartment — full kitchen, bathroom, heating. Self-contained. Takes time to build and resources to run.

Container analogy: A hotel room — you share the building's plumbing and electricity. Your room is yours. Check in within milliseconds.

Key numbers: Containers start in milliseconds vs minutes for VMs. 10–100× smaller. Run hundreds of containers where you might run only 20 VMs.

Why it matters: Same container image runs identically on dev laptop, CI/CD pipeline, and any cloud — eliminating 'works on my machine' forever.

Docker: The Container Engine That Changed Everything

- Released 2013 — made containers mainstream by wrapping complex Linux tech in a clean developer interface

Image: Read-only template defining what's inside a container. The cookie cutter. Stored in a Registry.

Container: A running instance of an image. Isolated, ephemeral, lightweight. The cookie baked from the cutter.

Dockerfile: Text file with step-by-step build instructions. The recipe.

Registry: Storage for images. Docker Hub (public). AWS ECR, GCP GCR, Azure ACR (private/enterprise).

Volume: Persistent storage that outlives a container. Used for model weights, training data, and logs.

💡 *docker run replaces pages of environment setup documentation. One command, reproducible result, every time.*

Kubernetes (K8s): Container Orchestration at Scale

- Docker alone works for one machine + a handful of containers.
- For 200 containers across 15 servers with auto-scaling, failures, and rolling deployments → you need Kubernetes.

Origin: Open-source, originally built by Google, released 2014. Now maintained by the CNCF (Cloud Native Computing Foundation).

Pod: Smallest deployable unit. Usually contains one container — your ML model API.

Deployment: Manages a set of identical Pods. Ensures the right number are always running.

Service: Stable network endpoint to access Pods, even as individual Pods come and go.

HPA: Horizontal Pod Autoscaler — auto-scales Pod count based on CPU/memory or custom metrics.

Namespace: Virtual cluster within a cluster — separates dev, staging, and production environments.

Containers vs Virtual Machines — Side-by-Side

Feature	Virtual Machines	Containers
Size	GBs per VM	MBs per container
Boot time	Minutes	Milliseconds
Isolation	Strong (full OS boundary)	Good (namespace-based)
Overhead	High — full guest OS	Minimal — shared host kernel
Portability	Medium	Excellent
Primary use case	Full OS environments, legacy apps	Microservices, ML APIs, CI/CD
Best for DS when...	Need full OS control or strong isolation	Reproducible ML experiments, model serving

💡 *Use VMs when you need full OS control or strong isolation. Use containers for everything else in modern data science.*

Containerising an ML Pipeline (1/2) – Project Structure

- Standard layout for a containerised ML training workflow:
- project/
 - └─ Dockerfile ← Build instructions for the environment
 - └─ requirements.txt ← Pinned Python dependency versions
 - └─ train_model.py ← Your ML training script

Dockerfile key instructions:

- FROM python:3.11-slim ← base image with Python pre-installed
- COPY requirements.txt . ← copy deps BEFORE code (enables caching)
- RUN pip install -r requirements.txt
- ENV PYTHONUNBUFFERED=1 ← ensure real-time log output
- CMD ["python", "train_model.py"]

Containerising an ML Pipeline (2/2) – Golden Rules

Rule 1 – Pin versions: requirements.txt must have exact versions (e.g. scikit-learn==1.4.2). Without this, builds on different days may install different versions, giving different model results.

Rule 2 – Set RANDOM_SEED: Setting a seed (e.g. RANDOM_SEED = 42) in training code ensures every container run produces identical results. Reproducibility is the whole point.

Rule 3 – Save the scaler: Always save the feature scaler alongside the model file. At inference time, new data must be scaled with the exact same scaler used in training.

Run commands: `docker build -t mymodel .` → `docker run --rm mymodel`

In cloud: Push image to AWS ECR / Docker Hub → deploy on AWS ECS, GCP Cloud Run, or Azure AKS

 *Same code + same environment = same result, always. That is the promise of containerised ML.*

Key Takeaways

Lesson 3 — Virtualization & Containerization

1

VMs abstract the hardware. Containers abstract the OS. Both solve the 'it works on my machine' problem, at different levels.

2

Type 1 hypervisors (KVM, Hyper-V, ESXi) power every major public cloud. Type 2 (VirtualBox) is for local dev and testing.

3

Docker = portable, reproducible environments. Kubernetes = manage hundreds of containers at scale with auto-scaling and self-healing.

4

Containerisation is the most important DevOps skill for a modern data scientist. One Dockerfile replaces pages of setup docs.

Session Summary — What You Learned Today

Lesson	Core Concept	Remember This
Lesson 1: Introduction	Cloud = on-demand, scalable, measured compute	IaaS/PaaS/SaaS; NIST 5 characteristics; AWS/Azure/GCP
Lesson 2: Architecture	Cloud = system of interconnected components	VPC, load balancer, CDN, API gateway, IAM — every ML system uses these
Lesson 3: Virtualisation	VMs abstract hardware; containers abstract OS	Docker = portability; Kubernetes = scale

Key Formulas & Pricing Models

- CLOUD STORAGE COST FORMULA:
- $\text{Monthly Storage Cost} = \text{storage_cost_per_gb} \times \text{total_gb}$
- $\text{Billable Egress} = \max(0, \text{total_egress_gb} - \text{free_tier_gb})$
- $\text{Monthly Egress Cost} = \text{billable_egress} \times \text{egress_rate}$
- $\text{Total Monthly Cost} = \text{Storage Cost} + \text{Egress Cost}$

- WEIGHTED LOAD BALANCER DISTRIBUTION:
- $\text{Server share} = \text{server_weight} / \text{sum_of_all_weights} \times \text{total_requests}$
- Example: Weight-2 server, total weight=5, 1000 requests:
- $\rightarrow 2/5 \times 1000 = 400$ requests routed to that server

Key insight: Egress fees often EXCEED storage costs. Always factor in outbound data transfer when choosing a provider.



Coming Up — Day 02: Cloud Storage & Data Lakes

- How Netflix, Airbnb, and Spotify store petabytes cost-effectively
- AWS S3 — deep dive into object storage design
- Delta Lake — ACID transactions on data lakes
- Apache Parquet — the columnar format powering modern analytics
- boto3 — the AWS Python SDK — hands-on code

- Recommended preparation:
 - → Sign up for AWS Free Tier: <https://aws.amazon.com/free/>
 - → Play with Docker (browser-based): <https://labs.play-with-docker.com/>
 - → Play with Kubernetes: <https://labs.play-with-k8s.com/>



Essential Resources

Standards & Cloud

- STANDARDS & DOCUMENTATION
- NIST SP 800-145 (cloud definition)
- csrc.nist.gov/publications
- CNCF (Kubernetes home): cncf.io
- GDPR text: gdpr-info.eu
- CLOUD FREE TIERS
- AWS Free Tier: aws.amazon.com/free
- GCP Free (\$300 credit): cloud.google.com/free
- Azure for Students: azure.microsoft.com
- HANDS-ON LABS
- Play with Docker: labs.play-with-docker.com
- Play with K8s: labs.play-with-k8s.com

Reading & Platforms

- PRICING CALCULATORS
- AWS: calculator.aws
- GCP: cloud.google.com/storage/pricing
- Azure: azure.microsoft.com/pricing
- DEEPER READING
- 12-Factor App: 12factor.net
- AWS Well-Architected: aws.amazon.com/architecture
- The Kubernetes Book — Nigel Poulton
- scikit-learn docs: scikit-learn.org
- DATA SCIENCE PLATFORMS
- Databricks: databricks.com
- Weights & Biases: wandb.ai
- AWS SageMaker: aws.amazon.com/sagemaker

Questions?

☁ Cloud Computing & Big Data Analytics

Day 01 — Introduction to Cloud Computing

Digital Science Graduate Programme